

Project #5 – FreeRTOS Task Management for PWM Motor Control

Title and Date

Document name: Project #5 – FreeRTOS Task Management for PWM Motor Control

Document reference: Thies, Nate.cmpe311.fall25.Project#5-TaskMgmtFreeRTOS

Date of publication: 12/7/2025

LEAD ENGINEER: Nate Thies, University of Maryland Baltimore County

STAKEHOLDERS: Prof. Kidd, Instructor, University of Maryland Baltimore County; MD
Safwan Zaman, Teaching Assistant, University of Maryland Baltimore County

High-Level Description

This project re-implements the variable-duty-cycle motor control system from Project #4 using FreeRTOS for task management instead of the custom cyclic executive (CE). The PWM waveform used to drive the DC motor is still generated by Timer2 in an interrupt service routine (ISR) on digital pin D9, while a pushbutton allows the user to step through discrete duty-cycle levels. The existing serial user interface and dual LED blink tasks from Project #2/4 are preserved as separate FreeRTOS tasks, demonstrating how a real-time operating system can manage multiple asynchronous activities on the Elegoo/Arduino UNO R3.

Description

In Project #4, the system used a non-preemptive cyclic executive (CE) to schedule tasks for serial input, LED blinking, button handling, and PWM motor control. The PWM signal for the motor was generated using Timer2 in CTC mode at 250 Hz, with an ISR controlling the duty cycle on D9. A pushbutton on D4 allowed the user to cycle the motor speed through an eight-step pattern (0/8, 2/8, 4/8, 6/8, 8/8, 6/8, 4/8, 2/8, 0/8) while the CE maintained responsiveness for the LED and serial tasks.

Project #5 keeps the same hardware, PWM mechanism, and user-visible behavior but replaces the CE with FreeRTOS task management APIs. The serial user interface, LED blink functions, button handling, and motor duty-cycle logic each run in their own FreeRTOS tasks, which cooperate through non-blocking code and real-time operating system primitives. A binary semaphore is introduced to coordinate the sharing of the button state between the button detection task and the motor duty-cycle task, satisfying the extra-credit objective. Overall, this design gives hands-on experience in migrating an existing embedded application from a custom task manager into a standard RTOS environment while preserving the functional behavior of the previous lab.

References

- Elegoo UNO R3 (ATmega328P) datasheet
- IRLZ34N MOSFET datasheet
- 1N4004 rectifier diode datasheet
- Arduino Timer2 documentation

- CMPE311 Project #2 – Task Management (Cyclic Executive)
- CMPE311 Project #4 – AddDutyCycle (PWM Motor Control)

Definitions and Abbreviations

PWM: Pulse Width Modulation

ISR: Interrupt Service Routine

MOSFET: Metal-Oxide-Semiconductor Field Effect Transistor

CE: Cyclic Executive

RTOS: Real-Time Operating System

Duty Cycle: Fraction of one PWM period for which the signal remains HIGH

Conventions

- Must, shall, will – mandatory requirement
- May – optional behavior
- Informative – explanatory content only
- Customer requirements begin with C.#
- High-level technical requirements begin with HL.#
- Test requirements begin with T.#

Customer Requirements

1. C.1 The user must be able to control the speed of the DC motor.
2. C.1.1 Speed control shall be performed through a single pushbutton.
3. C.2 The system shall step through discrete speed levels in a predictable pattern.
4. C.2.1 Steps shall follow the sequence: $0/8 \rightarrow 2/8 \rightarrow 4/8 \rightarrow 6/8 \rightarrow 8/8 \rightarrow 6/8 \rightarrow 4/8 \rightarrow 2/8 \rightarrow 0/8$.
5. C.3 The system must operate on an Elegoo/Arduino UNO R3 development board.
6. C.4 The system must remain responsive, with no blocking delays, while the motor is running.
7. C.5 The system shall operate safely with an externally powered motor.
8. C.6 The FreeRTOS-based implementation shall behave identically to Project #4 from the user's point of view (same button sequence and visible behavior).

High-Level Technical Requirements

9. HL.1 The system shall generate PWM on digital pin D9 using hardware Timer2.
10. HL.1.1 Timer2 shall operate at a PWM frame frequency of 250 Hz using an 8-slot PWM frame.
11. HL.2 The system shall use an IRLZ34N N-channel MOSFET as a low-side switch for the motor.
12. HL.2.1 The MOSFET gate shall use a 330–470 Ω series resistor and ≥ 4.7 k Ω pull-down resistor.

13. HL.3 The motor shall be powered from an 8×AA battery pack, electrically isolated from the Arduino 5 V logic supply but sharing a common ground.
14. HL.4 A 1N4004 diode shall be placed across the motor terminals to suppress inductive voltage spikes.
15. HL.5 The embedded firmware shall run under FreeRTOS task management instead of a cyclic executive.
16. HL.5.1 Existing tasks from Project #2/4 (serial UI, LED blinks, button handling, motor duty control) shall be implemented as separate FreeRTOS tasks with no use of delay().
17. HL.6 The button state shall be debounced in software and processed asynchronously.
18. HL.7 (Extra-credit requirement) The sharing of button press events between the button detection task and the motor duty-cycle task shall use a FreeRTOS synchronization primitive (binary semaphore).

Testing and Validation Requirements

19. T.1 Verify pushbutton operation.
20. T.1.1 System shall print the current duty-cycle step to the Serial Monitor each time the button is pressed, and the sequence shall match C.2.1.
21. T.2 Verify PWM generation on D9.
22. T.2.1 A test LED on D9 (or oscilloscope) shall show distinct brightness / duty levels for each step in the sequence.
23. T.3 Verify motor operation with the MOSFET driver.
24. T.3.1 The motor shall be off at 0/8 duty and increase speed as the duty-cycle steps increase, then decrease back to off following the pattern in C.2.1.
25. T.4 Validate FreeRTOS task responsiveness.
26. T.4.1 LED blink tasks and the serial UI shall remain responsive while the motor PWM and button handler tasks are running.
27. T.5 Validate semaphore-based event sharing (extra-credit).
28. T.5.1 The motor duty-cycle task shall only advance when the button task signals a debounced press via the binary semaphore.

Design

Equipment Required

- Elegoo / Arduino UNO R3 (ATmega328P)
- IRLZ34N N-channel MOSFET
- 1N4004 diode
- DC motor (3–12 V)
- 8×AA battery pack (~12 V)
- 330–470 Ω resistor (MOSFET gate series)
- 4.7 k Ω resistor (MOSFET gate pull-down)
- Two LEDs with ~220 Ω series resistors
- Momentary pushbutton
- Breadboard and jumper wires

Circuit Design (Text-Only Schematic)

Power Path:

- Battery pack + (~ 12 V) to motor + terminal.
- Motor – terminal to MOSFET Drain.
- MOSFET Source to common ground.
- Battery pack – to common ground.
- Arduino GND to common ground.

MOSFET Gate:

- Arduino D9 to MOSFET gate through $330\text{--}470\ \Omega$ series resistor.
- MOSFET gate to ground through $4.7\ \text{k}\Omega$ pull-down resistor.

Flyback Diode:

- 1N4004 cathode (striped end) to motor +.
- 1N4004 anode to motor –.

Pushbutton and LEDs:

- Pushbutton between Arduino D4 and ground, with D4 configured as INPUT_PULLUP.
- LED1: D2 $\rightarrow \sim 220\ \Omega$ resistor $\rightarrow$ LED $\rightarrow$ ground.
- LED2: D3 $\rightarrow \sim 220\ \Omega$ resistor $\rightarrow$ LED $\rightarrow$ ground.

Figure 1 (to be inserted): Motor driver schematic using IRLZ34N MOSFET and 1N4004 diode.

Software Architecture

Timer2 ISR (PWM Engine):

- Timer2 is configured in CTC mode with $\text{OCR2A} = 124$ and prescaler 64 to generate a 2 kHz tick.
- An 8-slot PWM frame is implemented in the ISR using a local counter and a global `g_pwmOnTicks` variable to determine how many ticks per frame the D9 output remains HIGH.

Duty-Cycle State Machine:

- The motor duty cycle is encoded in `dutyTable[] = {0, 2, 4, 6, 8, 6, 4, 2, 0}`.
- A FreeRTOS task `vTaskMotorDuty` blocks on a binary semaphore; each time it receives a button event, it advances the index into `dutyTable` and updates `g_pwmOnTicks`.

Button Handler Task:

- `vTaskButton` polls the button at 5 ms intervals and debounces via a time-based filter.
- On a debounced rising edge (released to pressed), it calls `xSemaphoreGive` on `xButtonPressSem` to signal a press to the motor duty-cycle task.

Serial UI Task:

- `vTaskSerialInput` wraps the non-blocking line editor and state machine handling the

'Which LED?' / 'What interval?' dialogue from earlier projects.

LED Blink Tasks:

- vTaskBlinkLED1 and vTaskBlinkLED2 toggle LEDs using millis()-based timing and vTaskDelay for cooperative scheduling.

Figure 2 (to be inserted): FreeRTOS task diagram showing ISR, Button Task, Motor Duty Task, Serial UI Task, and LED Tasks.

Code

```
None
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <semphr.h>

// ----- Pins -----

static const uint8_t LED_PIN_1 = 2; // LED1 on D2
static const uint8_t LED_PIN_2 = 3; // LED2 on D3
static const uint8_t PWM_PIN = 9; // Timer2 PWM output driving MOSFET
gate
static const uint8_t BUTTON_PIN = 4; // Pushbutton, active LOW
(INPUT_PULLUP)

// ----- Timing Model -----
// (Copied from Project #2/#4 for LED blinking behavior)

static volatile unsigned long periodMs_LED1 = 1000; // default 1s period
static volatile unsigned long periodMs_LED2 = 1000; // default 1s period

static unsigned long lastToggle_LED1 = 0;
static unsigned long lastToggle_LED2 = 0;

static bool state_LED1 = LOW;
static bool state_LED2 = LOW;

// ----- Serial UI State -----

static const size_t LINE_MAX = 32;
static char lineBuf[LINE_MAX];
static size_t lineLen = 0;

enum InputState : uint8_t {
```

```

    ASK_LED = 0,
    ASK_INTERVAL = 1
};

static InputState uiState = ASK_LED;
static int          selectedLED = -1; // 1 or 2 after a valid LED entry

// ----- Helpers -----

static inline unsigned long halfPeriodClamp(unsigned long period) {
    unsigned long hp = period / 2;
    return (hp == 0 ? 1UL : hp);
}

static void promptAskLED() {
    Serial.println(F("What LED? (1 or 2)"));
}

static void promptAskInterval() {
    Serial.println(F("What interval (in msec)?"));
}

static bool isAllDigits(const char* s) {
    if (!s || !*s) return false;
    for (const char* p = s; *p; ++p) {
        if (*p < '0' || *p > '9') return false;
    }
    return true;
}

// Process a completed input line (no CR/LF included).
static void processLine(const char* line) {
    switch (uiState) {
        case ASK_LED: {
            if ((strcmp(line, "1") == 0) || (strcmp(line, "2") == 0)) {
                selectedLED = atoi(line);
                uiState = ASK_INTERVAL;
                promptAskInterval();
            } else {
                Serial.println(F("Invalid LED. Enter 1 or 2.));
                promptAskLED();
            }
        }
        } break;
    }

```

```

    case ASK_INTERVAL: {
        if (!isAllDigits(line)) {
            Serial.println(F("Invalid interval. Must be a positive integer in
milliseconds."));
            promptAskInterval();
            return;
        }

        unsigned long val = strtoul(line, nullptr, 10);
        if (val == 0) {
            Serial.println(F("Invalid interval. Must be > 0."));
            promptAskInterval();
            return;
        }

        if (selectedLED == 1) {
            periodMs_LED1 = val;
            Serial.print(F("LED1 period set to "));
            Serial.print(periodMs_LED1);
            Serial.println(F(" ms (ON/OFF ~half each)."));
        } else if (selectedLED == 2) {
            periodMs_LED2 = val;
            Serial.print(F("LED2 period set to "));
            Serial.print(periodMs_LED2);
            Serial.println(F(" ms (ON/OFF ~half each)."));
        } else {
            Serial.println(F("Internal state error; resetting prompt."));
        }

        uiState = ASK_LED;
        selectedLED = -1;
        promptAskLED();
    } break;
}

// ----- PWM / Timer2 -----

// 8-slot PWM: each TIMER2 COMPA tick occurs at 2 kHz.
// 8 ticks per PWM frame → 250 Hz PWM frequency.
:contentReference[oaicite:15]{index=15}
static const uint8_t PWM_MAX_TICKS = 8;

```

```

// Number of "ON" ticks per frame, 0..8. Written by motor-duty task, read
by ISR.
volatile uint8_t g_pwmOnTicks = 0;

// Set up Timer2 in CTC mode with OCR2A = 124 and prescaler 64 to get 2
kHz.
static void setupTimer2PWM() {
    cli();

    pinMode(PWM_PIN, OUTPUT);
    digitalWrite(PWM_PIN, LOW);

    // Clear control registers
    TCCR2A = 0;
    TCCR2B = 0;

    // CTC mode: WGM21 = 1, WGM20 = 0, WGM22 = 0 → Mode 2
    TCCR2A |= (1 << WGM21);

    // Compare value: (16 MHz / (64 * 2000 Hz)) - 1 = 124
    OCR2A = 124;

    // Enable Compare Match A interrupt
    TIMSK2 |= (1 << OCIE2A);

    // Prescaler = 64 → CS22=1, CS21=0, CS20=1
    TCCR2B |= (1 << CS22) | (1 << CS20);

    sei();
}

// Timer2 Compare Match A ISR: generate PWM on D9 using 8-slot frame.
ISR(TIMER2_COMPA_vect) {
    static uint8_t s_pwmCounter = 0;

    s_pwmCounter++;
    if (s_pwmCounter >= PWM_MAX_TICKS) {
        s_pwmCounter = 0;
    }

    uint8_t onTicks = g_pwmOnTicks; // 0..8

    if (s_pwmCounter < onTicks) {
        digitalWrite(PWM_PIN, HIGH);
    }
}

```



```

    } else {
        digitalWrite(PWM_PIN, LOW);
    }
}

// ----- FreeRTOS: Button & Motor -----

// Duty pattern copied from Project #4 spec: 0/8 → 2/8 → 4/8 → 6/8 → 8/8
// → 6/8 → 4/8 → 2/8 → 0/8. :contentReference[oaicite:16]{index=16}
static const uint8_t DUTY_TABLE_LEN = 9;
static const uint8_t dutyTable[DUTY_TABLE_LEN] = {0, 2, 4, 6, 8, 6, 4, 2,
0};

static uint8_t g_dutyIndex = 0; // index into dutyTable

// Binary semaphore shared between button task and motor-duty task
static SemaphoreHandle_t xButtonPressSem = NULL;

// Button polling parameters
static const TickType_t BUTTON_TASK_PERIOD = pdMS_TO_TICKS(5); // 5 ms
poll
static const uint8_t    DEBOUNCE_COUNT    = 4;                // 4*5 ms
= 20 ms

// ----- Legacy Tasks (now wrapped) -----

// Task 1: Non-blocking serial input reader + UI state machine.
static void taskSerialInput(void) {
    while (Serial.available() > 0) {
        char c = (char)Serial.read();

        if (c == '\r') {
            continue; // ignore CR, wait for LF
        } else if (c == '\n') {
            lineBuf[lineLen] = '\0';
            if (lineLen > 0) {
                processLine(lineBuf);
            }
            lineLen = 0;
        } else if (c == 8 || c == 127) { // Backspace or Delete
            if (lineLen > 0) {
                lineLen--;
            }
        } else {

```

```

        if (lineLen < (LINE_MAX - 1)) {
            lineBuf[lineLen++] = c;
        }
        // else: silently drop extra chars
    }
}

// Task 2: Blink LED1 according to its configured period.
static void taskBlinkLED1(void) {
    unsigned long now = millis();
    unsigned long hp  = halfPeriodClamp(periodMs_LED1);

    if ((now - lastToggle_LED1) >= hp) {
        state_LED1 = !state_LED1;
        digitalWrite(LED_PIN_1, state_LED1 ? HIGH : LOW);
        lastToggle_LED1 = now;
    }
}

// Task 3: Blink LED2 according to its configured period.
static void taskBlinkLED2(void) {
    unsigned long now = millis();
    unsigned long hp  = halfPeriodClamp(periodMs_LED2);

    if ((now - lastToggle_LED2) >= hp) {
        state_LED2 = !state_LED2;
        digitalWrite(LED_PIN_2, state_LED2 ? HIGH : LOW);
        lastToggle_LED2 = now;
    }
}

// ----- FreeRTOS Task Wrappers -----

// FreeRTOS task: serial input
static void vTaskSerialInput(void* pvParameters) {
    (void) pvParameters;
    for (;;) {
        taskSerialInput();
        vTaskDelay(pdMS_TO_TICKS(1)); // yield frequently
    }
}

// FreeRTOS task: LED1 blinker

```

```

static void vTaskBlinkLED1(void* pvParameters) {
    (void) pvParameters;
    for (;;) {
        taskBlinkLED1();
        vTaskDelay(pdMS_TO_TICKS(1));
    }
}

// FreeRTOS task: LED2 blinker
static void vTaskBlinkLED2(void* pvParameters) {
    (void) pvParameters;
    for (;;) {
        taskBlinkLED2();
        vTaskDelay(pdMS_TO_TICKS(1));
    }
}

// FreeRTOS task: Button debouncer and event generator
static void vTaskButton(void* pvParameters) {
    (void) pvParameters;

    bool lastRaw      = false;
    bool debounced    = false;
    bool lastDebounced = false;
    uint8_t stableCnt = 0;

    for (;;) {
        bool rawPressed = (digitalRead(BUTTON_PIN) == LOW); // active LOW

        if (rawPressed == lastRaw) {
            if (stableCnt < 255) stableCnt++;
        } else {
            stableCnt = 0;
            lastRaw    = rawPressed;
        }

        if (stableCnt >= DEBOUNCE_COUNT && rawPressed != debounced) {
            debounced = rawPressed;

            // Rising edge of a debounced press (released -> pressed):
            if (debounced && !lastDebounced) {
                // Signal a button-press event via semaphore
                xSemaphoreGive(xButtonPressSem);
            }
        }
    }
}

```

```

        lastDebounced = debounced;
    }

    vTaskDelay(BUTTON_TASK_PERIOD);
}

// FreeRTOS task: Motor duty-cycle state machine
static void vTaskMotorDuty(void* pvParameters) {
    (void) pvParameters;

    // Initialize to first duty entry (index 0 → 0/8 → motor off)
    g_dutyIndex    = 0;
    g_pwmOnTicks   = dutyTable[g_dutyIndex];

    Serial.print(F("Initial duty: "));
    Serial.print(dutyTable[g_dutyIndex]);
    Serial.println(F("/8"));

    for (;;) {
        // Block until a debounced button press event occurs
        if (xSemaphoreTake(xButtonPressSem, portMAX_DELAY) == pdTRUE) {
            // Advance through the duty pattern
            g_dutyIndex++;
            if (g_dutyIndex >= DUTY_TABLE_LEN) {
                g_dutyIndex = 0;
            }

            g_pwmOnTicks = dutyTable[g_dutyIndex];

            Serial.print(F("Button press -> duty "));
            Serial.print(dutyTable[g_dutyIndex]);
            Serial.println(F("/8"));
        }
    }
}

// ----- Arduino setup/loop -----

void setup() {
    pinMode(LED_PIN_1, OUTPUT);
    pinMode(LED_PIN_2, OUTPUT);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
}

```

```

digitalWrite(LED_PIN_1, LOW);
digitalWrite(LED_PIN_2, LOW);

Serial.begin(9600);
while (!Serial) { /* UNO returns immediately; placeholder for
compatibility */ }

Serial.println(F("System ready (FreeRTOS)."));
promptAskLED();

// Configure PWM on D9 via Timer2 ISR
setupTimer2PWM();

// Create semaphore for button press events
xButtonPressSem = xSemaphoreCreateBinary();
if (xButtonPressSem == NULL) {
    Serial.println(F("ERROR: Failed to create button semaphore."));
}

// Create FreeRTOS tasks
xTaskCreate(vTaskSerialInput, "SerialIn",
           256, NULL, 2, NULL);
xTaskCreate(vTaskBlinkLED1, "LED1",
           128, NULL, 1, NULL);
xTaskCreate(vTaskBlinkLED2, "LED2",
           128, NULL, 1, NULL);
xTaskCreate(vTaskButton, "Button",
           128, NULL, 3, NULL); // higher priority to capture
presses
xTaskCreate(vTaskMotorDuty, "Motor",
           128, NULL, 2, NULL);

// The FreeRTOS scheduler is started by the Arduino_FreeRTOS core
// after setup() returns, so loop() is left empty.
}

void loop() {
    // Not used; all work is done in FreeRTOS tasks.
}

```

Testing

Procedure

1. Button / Serial Check:

- Power the Arduino from USB only (motor disconnected).
- Open the Serial Monitor at 9600 baud and verify that the system prints the initial prompt and responds correctly to LED period commands.

2. PWM LED Test:

- Connect a test LED with series resistor from D9 to ground.
- Press the pushbutton repeatedly and observe the LED brightness stepping through the expected pattern.

3. Motor Driver Integration:

- Connect the external battery pack, motor, MOSFET, and diode as described in the Circuit Design.
- Confirm that at 0/8 duty the motor is completely off and that speed increases and then decreases as the duty-cycle sequence advances with each button press.

4. FreeRTOS Responsiveness:

- While cycling motor speeds, adjust LED periods via the serial UI and verify that LED blinking and serial input remain responsive.

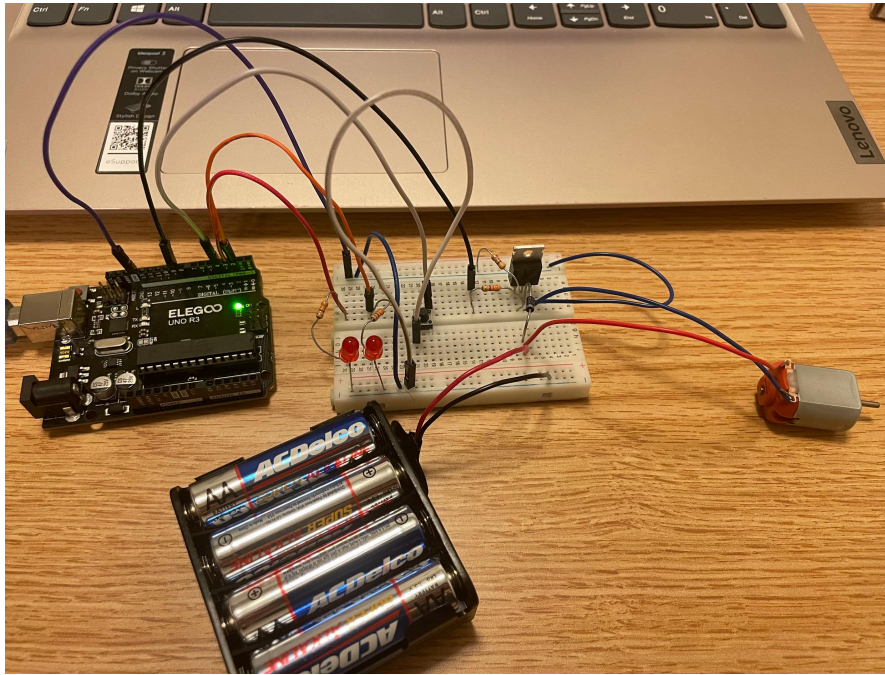
5. Semaphore Validation (Extra-Credit):

- Use the existing debug messages in vTaskMotorDuty to confirm that duty changes only occur in response to debounced button press events reported by the button task via the binary semaphore.

Demo Video

<https://drive.google.com/file/d/1kNya7uTkXK7SA0aCIndl4y2oAfaji5ex/view?usp=sharing>

Picture



Results

The button was successfully debounced, with each press advancing the duty cycle exactly one step and no observable contact bounce in the Serial Monitor output. The Timer2 ISR produced a stable PWM signal on D9, and the test LED clearly showed distinct brightness levels as the duty cycle moved through the sequence. When connected to the MOSFET driver and external battery pack, the DC motor remained off at 0/8 duty, increased in speed at 2/8, 4/8, and 6/8, reached full speed at 8/8, and then decreased back to off following the specified pattern. The serial UI and LED blink tasks remained responsive throughout motor operation, demonstrating that the FreeRTOS scheduler correctly multiplexed tasks without blocking. Debug messages confirmed that the motor duty-cycle task only advanced when signaled by the button task via the binary semaphore, satisfying the extra-credit synchronization requirement.